Tyler Harris
599 Edge AI and Robotics
Module 3 Face and Eye Recognition

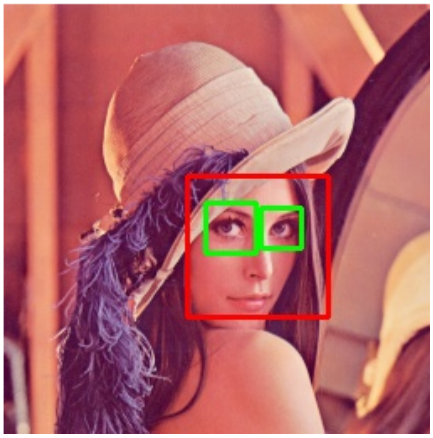
Overview:
The purpose of this lab is to introduce face and eye detection for the purpose of redacting faces for privacy, as well as the redaction process. Redaction is performed on both a CPU and GPU implementation, with the performance difference timed as with previous labs.

Face and Eye Detection:

- In order to redact a face there first needs to be a bounding box surrounding it.
- The cv2.rectangle method can draw a rectangle over an image but requires the start and end points for the rectangle
- Using a haarcascade classifier, a bounding box around the face can be found and the coordinates can then be passed to the cv2.rectangle to actually draw the bounding box.

Before Face and Eye Detection:

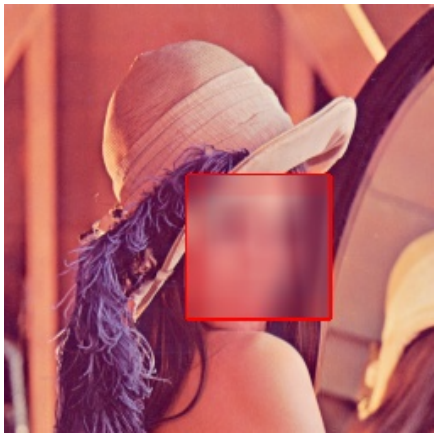
After:



Redacting Image (CPU):

The process is largely the same as the code for simply drawing the bounding boxes:

- A haarcascade classifier is used to identify the face in the image (no eyes)
- The coordinates of the bounding box are given to the cv2.rectangle function
- The cv2.GuassianBlur is used to actually redact the face on the identified region

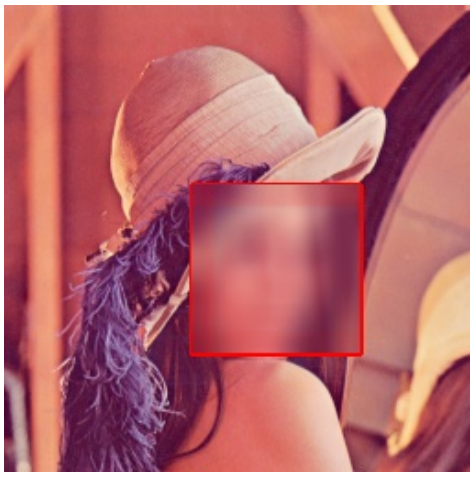


The CPU implementation took a total of 184ms.

Redacting Image (GPU):

The difference with GPU implementation is that the CPU no longer processes the portion where the haarcascade is used to find the face in the image:

- The image is stored as a cuda_GpuMat()
- The image is uploaded to the GPU and converted to grayscale
- Again the haarcascade classifier is used to detect the faces
- The grayscale image is moved back to the host
- The CPU still must perform the Guassian blur inside the bounding box



The GPU implementation actually took significantly longer, totaling 564ms for the sample image provided.

Seeing that the GPU took significantly longer, I tried again with an image that contained many faces.

CPU Result:

```
CPU times: user 568 ms, sys: 36 ms, total: 604 ms
Wall time: 268 ms
```

It is interesting to note that the CPU missed many faces, this is likely because the haarcascade from what I understand is an old machine learning approach and certain faces are harder to differentiate when they are out of focus or somewhat monochromatic.

GPU Result:

```
CPU times: user 312 ms, sys: 48 ms, total: 360 ms
Wall time: 568 ms
```

It can be seen that the GPU is in fact faster in this case when it has more work to do. It is also a bit interesting to see that it caught an additional face the CPU missed.

Conclusion:
This lab served as a great introduction to the basics of how automatic facial recognition is performed using haarcascade algorithms for the bounding boxes and a blur can be applied to the discovered facial region. It was also interesting to see the differences in CPU and GPU implementation, although they are fairly minimal since the Python cuda abstraction simplifies so much.